DOCUMENT

06

Developer Guide

Setup, commands, conventions and troubleshooting

1. Prerequisites

Tool	Version	Needed for
Python	3.10 or newer (3.13 used)	backend and tests
Node.js	20 or newer (24 used)	Playwright end-to-end tests
WSL with Ubuntu	22.04	mutation testing on Windows (mutmut 3 needs `fork()`)
git	any	version control

2. Setup and run

```
pip install -e "backend[dev]"
python -m uvicorn invoice_app.api.main:app --app-dir backend/src --port 5000
```

On Windows, if you set `PYTHONPATH` yourself, separate entries with `;`.

3. Commands

Task	Command
Unit and API tests	`python -m pytest backend/tests -q`
End-to-end tests	`cd e2e && npm install && npx playwright install chromium && npm test`
Mutation tests	`wsl -d Ubuntu-22.04 -- bash scripts/run_mutation.sh <label>` (writes `docs/mutation/<label>.txt`)
Show surviving mutants	`scripts/show_survivors.sh`
Rebuild these PDFs	`python scripts/build_docs.py`

The Playwright config starts the server itself (`webServer`) on port 5000 and reuses one that is already running.

4. Conventions

- Business rules go in `domain/`. Routers translate HTTP to domain calls; `api/error_handling.py` maps domain errors to statuses.
- Follow SOLID (document 19): one reason to change per class, extend by adding classes, depend on small contracts, inject collaborators.
- Money is `Decimal`. Never `float`.

- All checks run before the first mutation. A refused attempt changes nothing.
- Usernames are compared with `strip().casefold()`.
- Use clear names; no one-letter variables and no boolean `flag` parameters.
- End-to-end tests use page objects and role-based locators (`getByRole`, `getByLabel`), never CSS or XPath.
- Every new rule or endpoint needs tests. Use `FakeInvoiceRepository` in unit tests.
- Never commit real credentials, tokens or customer data.

5. How to add or change an approval rule

1. Update the spec (`specs/001-invoice-approval-limits/spec.md`) and the table in `AGENTS.md`.
2. Write the failing tests first: policy test for the rule, a service test using `assert_refused_and_unchanged` for each refusal, and an API test for the status and message.
3. Add a **new rule class** to `approval_policy.py` (pure rules only, with a `refusal(attempt)` method) and list it in `standard_policy()`. Do not edit the existing rule classes. The order of the list is the order reasons are reported in; the decision table in document 02 shows the current order.
4. If a refusal needs a new message, assert the exact text.
5. Run pytest, then the mutation run, and triage every survivor (add a test, or record why it is equivalent).
6. Add a Playwright test only if the UI behaviour changes.

6. Writing tests that can fail

- A negative test must assert the **reason text** and that the **invoice is unchanged** (deep copy compared by equality) and that the repository saw **no update**.
- Fakes must behave like the real thing: `FakeInvoiceRepository` stores copies and records updates, and raises on unknown ids.
- For concurrency, use `SnapshotRepository` (snapshot reads, delayed saves). A test against a live-object repository passes even without the lock and proves nothing.
- Judge tests by mutation score, not line coverage.

7. Continuous integration

`.github/workflows/ci.yml` runs on pushes and pull requests to `main`:

Job	What it does
`unit-tests`	install, `pytest backend/tests -q`
`mutation-tests`	`scripts/run_mutation.sh ci`, uploads the report as an artifact
`e2e`	installs Chromium, runs the Playwright test

8. Landing changes

Land features in small steps, each commit saying what changed and why. A suggested sequence for this work is in document 13. In the pull request description every review finding gets exactly one verdict: **fixed** (say how), **dismissed** (give a reason) or **deferred** (give a reason and where it is tracked).

9. Troubleshooting

Symptom	Cause and fix
`mutmut` fails with a fork error on Windows	Run it in WSL via `scripts/run_mutation.sh`
Git Bash rewrites `/mnt/...` paths when calling `wsl`	Prefix the command with `MSYS_NO_PATHCONV=1`
mutmut reports ImportError for API tests	The mutation scope is the domain package only; `pyproject.toml` limits test selection to the domain tests
`python -m venv` fails with ensurepip error on a OneDrive path	Create the venv on a short local path (or use WSL)
Playwright cannot start the server	Check that port 5000 is free or start the server yourself and let it be reused
Everything resets after a restart	Expected: storage is in memory